Quiver Tap — Arquitetura de Observabilidade

Documento gerado em 11/08/2026 · Ambientes dev (:3002 / :5202) e prod (:3001) · Stack Elastic local

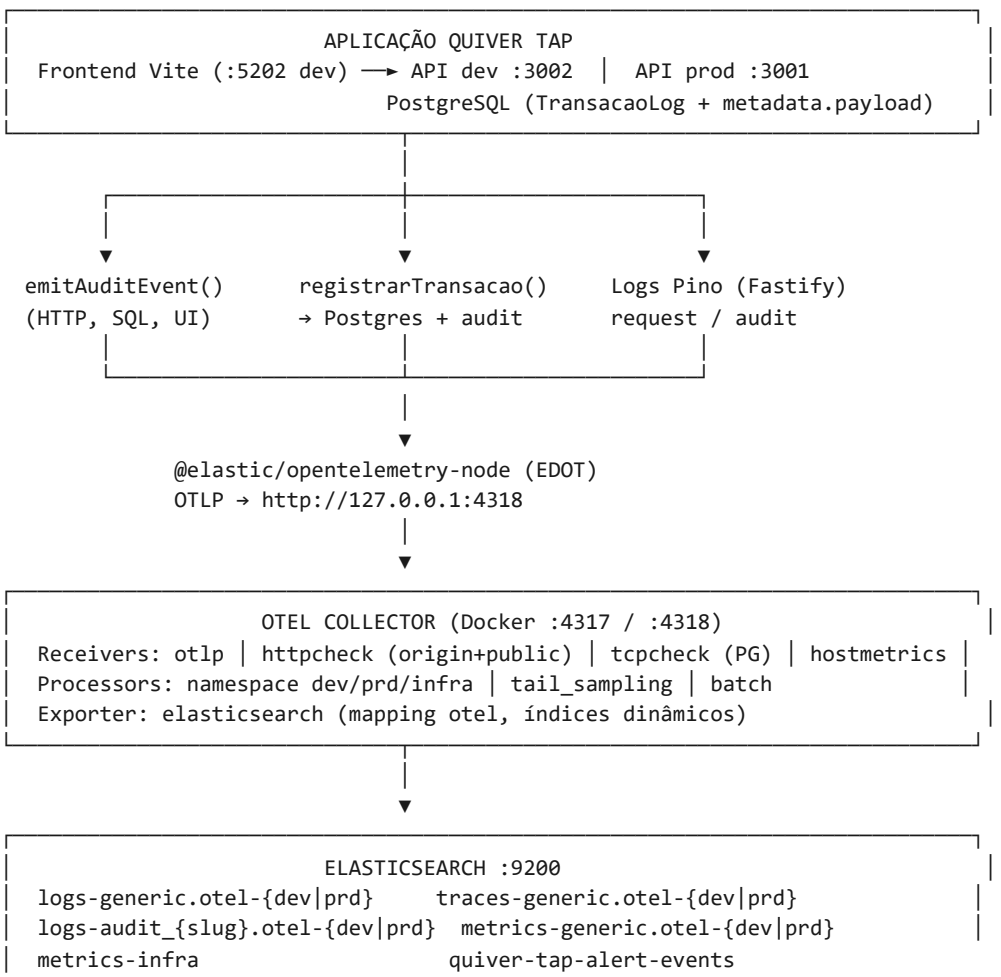
1. Visão geral

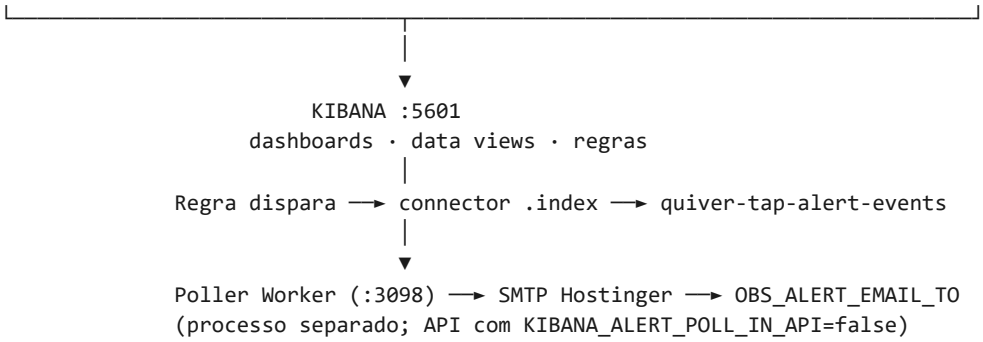
A observabilidade do Quiver Tap combina **OpenTelemetry (EDOT Node)**, um **OTel Collector**, **Elasticsearch** e **Kibana**. A aplicação emite logs, traces e métricas; audit e transações de negócio vão para índices separados por empresa (slug). Alertas do Kibana (licença Basic) são entregues por e-mail via connector `.index` + poller na API + SMTP.

Camadas

- **Aplicação** — API Fastify + hooks de audit + `registrarTransacao()`
- **Emissão** — Pino instrumentado + EDOT Node SDK → OTLP
- **Coleta** — OTel Collector (OTLP + httpcheck + tcpcheck + hostmetrics)
- **Armazenamento** — Elasticsearch (índices dinâmicos OTel)
- **Visualização** — Kibana (data views, dashboards, regras)
- **Alertas** — ES `quiver-tap-alert-events` → worker dedicado → e-mail

2. Diagrama de arquitetura





3. Fluxo de telemetria da aplicação

1. Requisição HTTP entra no Fastify; hooks resolvem contexto (slug, actor, env).
2. Mutações e erros geram `emitAuditEvent` (HTTP audit).
3. Operações de negócio chamam `registrarTransacao()` → grava `TransacaoLog` no Postgres e emite audit com `transacao.payload`.
4. Prisma extension audita SQL (`db.*`), com amostragem configurável em leituras.
5. Tudo passa pelo logger Pino singleton (instrumentado) → EDOT exporta logs, traces e métricas via OTLP.
6. Collector aplica namespace `dev` ou `prd` e indexa no Elasticsearch.

Atributos principais (audit / transação)

Campo	Exemplo / uso
`attributes.event.category`	`audit`
`attributes.event.action`	`transacao.quiosque_comanda_criada`, `http.post`, `db.update`
`attributes.cervejaria.slug`	`churrasquinho-do-thiago`
`attributes.transacao.payload`	Objeto transacionado (com redação de segredos)
`attributes.event.local_time`	Horário de negócio (America/Sao_Paulo)
`data_stream.dataset`	`audit_{slug}`
`data_stream.namespace`	`dev` ou `prd`

4. Infraestrutura monitorada

O collector coleta métricas de infra independentemente da API:

Receiver	Alvo	Intervalo
httpcheck	`/api/ready` origin (:3001/:3002), URLs públicas (tap/tapdev), ES, Kibana, UI prod	60s
tcpcheck	PostgreSQL prod (:5432) e dev (:5434) via `host.docker.internal`	60s
hostmetrics	CPU, memória, disco, filesystem, paging do host	60s

Métricas infra usam namespace `infra` → índice `metrics-infra`, separado das métricas de app.

5. Índices no Elasticsearch

Sinal	Padrão de índice	Conteúdo
Logs genéricos	`logs-generic.otel-{dev|prd}`	Requests Fastify, startup, poller
Audit / transações	`logs-audit_{slug}.otel-{dev|prd}`	Um índice lógico por empresa
Métricas app	`metrics-generic.otel-{dev|prd}`	OTel metrics da API
Métricas infra	`metrics-infra`	Uptime + host
Traces	`traces-generic.otel-{dev|prd}`	Traces amostrados
Alertas	`quiver-tap-alert-events`	Fila para e-mail

6. Alertas por e-mail (Kibana Basic)

Licença Basic não permite connectors nativos `.email` / `.webhook`. Fluxo alternativo:

```
Kibana Rule (ex.: API down, 5xx, event loop)
        │
        ▼
Connector .index  ──►  ES: quiver-tap-alert-events
        │
        ▼
startKibanaAlertPoller() no worker (intervalo ~30s)
        │
        ▼
handleKibanaAlertWebhook()  ──►  SMTP (Hostinger)  ──►  OBS_ALERT_EMAIL_TO
```

Worker de alertas

O poller roda em `backend/src/alert-poller-worker.ts` (health `:3098/health`), gerenciado por `npm run alert-poller:start` e `scripts/prod/ensure-running.ps1`. Estado persistente em `deploy/runtime/alert-poller-state.json`. A API só faz poll se `KIBANA_ALERT_POLL_IN_API=true`.

Outside-in e SLO

- URLs públicas no collector + Kibana Uptime + UptimeRobot (ver `outside-in.md`)

- SLI/SLO documentados em `sli-slo.yml`; dashboards SLO e Business no Kibana
- Relatório mensal: `npm run obs:report-slo`

Variáveis relevantes

Variável	Função
`OTEL_EXPORTER_OTLP_ENDPOINT`	`http://127.0.0.1:4318`
`ELASTIC_OTEL_NODE_ENABLE_LOG_SENDING`	`true` — export de logs Pino
`OTEL_SERVICE_NAME`	`quiver-tap-api-dev` / prod
`OBS_ALERT_EMAIL_TO`	Destinatário dos alertas
`KIBANA_ALERT_POLL_IN_API`	`false` — poll no worker, não na API
`KIBANA_ALERT_POLL_ENABLED`	`true` no worker
`KIBANA_ALERT_POLL_STATE_FILE`	Cursor/dedup persistente do poller
`ALERT_POLLER_HEALTH_PORT`	Default 3098
`KIBANA_ALERT_POLL_INTERVAL_MS`	Default 30000 (30s)

7. Segregação dev vs prod

	Dev	Prod
API	:3002	:3001
Frontend	Vite :5202	Estático via API
Banco	`cervejaria_dev`	`cervejaria`
Namespace ES	`*.otel-dev`	`*.otel-prd`
Service name	`quiver-tap-api-dev`	`quiver-tap-api`

Mesmo cluster Elasticsearch/Kibana local; separação por `data_stream.namespace`.

8. Limites conhecidos (não é 100% de tudo)

- **Traces:** tail sampling — erros, lentos (>500ms) e ~15% probabilístico; maioria das requisições normais não traceada.
- **Audit HTTP:** GET 2xx e rotas de polling/health/assets ignorados.
- **Transações com payload:** fluxos principais (quiosque, comandas, eventos, portal, caixa, estoque); nem toda rota CRUD.
- **Infra:** collector cobre origin, URLs públicas e TCP Postgres; probe externo (UptimeRobot) documentado.
- **Alertas:** latência de até ~30s (poller worker); depende de regras Kibana ativas e SMTP configurado.
- **Deploy:** fixes recentes de export audit OTel validados em dev; prod reflete após deploy autorizado.

Quiver Tap · Observabilidade · Repositório: deploy/observability/ · Collector: otel-collector-config.yml · Scripts: scripts/observability/